The Privacy–Utility Trade-off, Worked Through

One use case, from the business question to a signed-off privacy setting

Study notes

2026-10-23

A bank must train a targeting model with differential privacy (DP), and someone other than the engineer has to choose how much privacy to buy. This note works through that choice on a public dataset, from a business count and a sentence a privacy officer can sign to a short menu of options and a decision.

Table of contents

1 The use case 2

2 Step 1: state the business goal as a count 3

3 Step 2: turn the privacy setting into a sentence 4

4 Step 3: fix everything that must not be tuned on private data 5

5 Step 4: read the results, technical first 7

6 Step 5: check how much the data size matters 9

7 Step 6: present the menu and decide 10

8 Doing this in another business 12

9 Sources 12

How strict should a differential privacy (DP) guarantee be, and who decides? This note answers it for one use case, a bank that trains a targeting model, and ends with a decision that three people sign. You need basic probability to follow it, such as odds, averages and percentiles. Table 1 gives the question each section answers, and Table 2 lists the symbols.

Table 1: The question each section answers.

Section	The question it answers
Section 1	What does the bank want, and which public data stands in for its records?

Section	The question it answers
Section 2	Which count does the business care about, and what must a model beat?
Section 3	What does ε mean to a privacy officer, and which value does the policy allow?
Section 4	Which choices must be fixed before training, so that they leak nothing?
Section 5	Which technique is best at each privacy level, and what does privacy cost in customers?
Section 6	How does that cost change with the number of customers?
Section 7	Which option does the policy pick, and who signs it?
Section 8	How do the steps carry over to another business?
Section 9	Where do the data, the methods and the arguments come from?

Table 2: The symbols the note uses. Each is also explained where it first appears.

Symbol	Meaning
ε	the privacy parameter of DP: a smaller value means stronger privacy
e^{ε}	the most that seeing the model can multiply an attacker's odds by
δ	the small probability that the guarantee fails outright
V	what the bank gains when it reaches one eligible customer instead of an ineligible one
k	a difference in the number of eligible customers reached

1 The use case

A retail bank is about to launch a premium credit card for customers who earn more than $50,000 a year, and it wants to send the offer to them alone. The bank does not record income, but it holds other facts about each customer, such as age, education, occupation, hours worked and marital status. So the marketing team wants a model that predicts from those facts who is likely to be eligible.

An outside agency runs the campaign, so the model will leave the bank's secure environment. For that reason the bank's privacy policy requires DP training, so that nobody can use the model to learn much about any one customer. The policy does not say how strict the guarantee must be, and that choice is the subject of this note.

The bank's records are private, so we use a public dataset like them, called Adult. Adult holds 48,842 people from the 1994 US Current Population Survey. Each has these attributes and a label for income above $50,000 (UCI Machine Learning Repository). Its training file of 32,561 people plays the bank's customer records, which the model learns from. Its test file of 16,281 people plays the customers of the next campaign. One script computes every number below, code/notes/privacy_utility_tradeoff.py, and each step shows its lines. It runs in a few minutes on all of a machine's processor cores:

```
uv run python code/notes/privacy_utility_tradeoff.py
```

2 Step 1: state the business goal as a count

Engineers usually measure a model by its accuracy or its area under the curve (AUC), but neither tells the marketing team what it will get. The team cares about how many offers reach eligible customers, so this step fixes the number of offers and counts how many land well.

The bank can choose the number of offers from public data. The Census Bureau publishes the share of people who earn more than $50,000, about 24% in Adult. If the bank sends as many offers as it expects eligible customers, a perfect model wastes none. For the 16,281 campaign customers this rule gives 3,846 offers, the number of eligible customers among them, when the published share holds for the bank.

The bank sends the 3,846 offers to the customers with the highest scores. The business metric is the number of eligible customers among them, and every other offer is wasted. The script computes it with a short function:

```
def reached(scores: Array, y: npt.NDArray[np.int64], offers: int) -> int:
    """Eligible customers among the `offers` highest-scored; ties are broken by
     ↪  position, deterministically."""
    top = np.argsort(-scores, kind="stable")[:offers]
    return int(y[top].sum())
```

The metric needs a baseline, and the natural one is no model at all. If the bank sent the offers to customers picked at random, each would reach an eligible customer with probability 3,846 / 16,281, about 24%. So the expected number reached would be

$$3{,}846 \times \frac{3{,}846}{16{,}281} \approx 909. \tag{1}$$

Any model, private or not, has to beat 909 to be worth having. The script sets up the campaign in the same way:

```
df = practitioner_datasets.adult()
train, test = df.head(TRAIN_ROWS), df.slice(TRAIN_ROWS)
data = (features(train), labels(train), features(test), labels(test))
eligible = int(data[3].sum())
# Random offers reach an eligible customer at the base rate, eligible / campaign
 ↪  customers.
```

```
baseline = eligible * eligible / len(data[3])
```

The count also gives the money. Write V for what the bank gains when it reaches one eligible customer instead of an ineligible one. It is the chance that an eligible customer takes up the card times the card's margin, and the finance team estimates it. Every comparison below is a difference of k eligible customers reached, worth kV to the bank. So the note never needs a value for V, because the decision in Section 7 only compares kV with what the policy protects.

3 Step 2: turn the privacy setting into a sentence

DP is set by one number, ε, and a smaller ε means stronger privacy. A privacy officer cannot sign a bare number, so this step translates ε into an attacker's question. Suppose an attacker wants to know whether one person, say Alice, is a customer of the bank, and starts with some belief about it. DP guarantees that seeing the model can multiply the attacker's odds by at most e^{ε}.

The simplest case is an attacker who starts at even odds, a belief of 50%. The odds are then 1 to 1, and after the attacker sees the model they are at most e^{ε} to 1. Odds of e^{ε} to 1 are a probability of e^{ε} out of $1 + e^{\varepsilon}$, so the attacker's belief can rise to at most

$$\text{belief ceiling} = \frac{e^{\varepsilon}}{1 + e^{\varepsilon}}. \tag{2}$$

At $\varepsilon = 1$ the ceiling is $e/(1 + e)$, about 73%. At $\varepsilon = 0.1$ it is about 52.5%, so the model teaches the attacker almost nothing. At $\varepsilon = 3$ it is about 95%, a guarantee that still holds but is weak.

The privacy officer signs this sentence, and in our use case they choose a ceiling of 75%. In other words, an attacker who thinks it is a coin flip whether Alice is a customer must not end up more than 75% sure. To turn the sentence into a number, multiply out the fraction to get a bound on e^{ε}, then take the logarithm:

$$\frac{e^{\varepsilon}}{1 + e^{\varepsilon}} \le 0.75 \quad \Longleftrightarrow \quad e^{\varepsilon} \le 3 \quad \Longleftrightarrow \quad \varepsilon \le \ln 3 \approx 1.10. \tag{3}$$

The script holds both directions of this translation. The first function labels the top axis of every figure below with the ceiling, and `epsilon_for_belief(0.75)` returns the policy's limit of 1.10:

```
def belief_ceiling(eps: float, prior: float = 0.5) -> float:
    """Highest posterior that epsilon-DP allows an attacker who starts at `prior` that
     ↪  one customer is in the data."""
    odds = math.exp(eps) * prior / (1 - prior)
    return odds / (1 + odds)


def epsilon_for_belief(belief: float, prior: float = 0.5) -> float:
    """The largest epsilon whose belief ceiling, from `prior`, is `belief`."""
    return math.log((belief / (1 - belief)) / (prior / (1 - prior)))
```

The officer fixes the ceiling before anyone sees the model's results, because a ceiling chosen afterwards would drift to wherever the model does well. The training method below adds a second parameter, δ, the small probability that the guarantee fails outright. A common rule sets δ below one over the number of people, so that the guarantee is not expected to fail for anyone. For 32,561 customers this gives $\delta = 10^{-5}$, and we use that value throughout.

4 Step 3: fix everything that must not be tuned on private data

A DP guarantee covers what the training run does with the data, but not choices made earlier by looking at it. If an engineer tried twenty settings on the customer records and kept the best one, that choice would itself leak information. So every choice outside the training run comes from public information.

The preprocessing comes from the survey's public codebook, not from the data. It scales ages within 17 to 90 and years of education within 1 to 16. It scales weekly hours within 1 to 99, and capital gains and losses on a log scale up to the survey's published cap of 99,999. Each category, such as an occupation, becomes a 0-or-1 column taken from the codebook's list. So every customer's row lies in a known range, which DP needs, because that range bounds how much one customer can change the training.

The preprocessing function reads only these public ranges and lists, never a statistic of the data:

```
def features(df: pl.DataFrame) -> Array:
    """Map each row to [0, 1]^d using only the codebook; a missing category is a row of
     ↪ zeros."""
    cols: list[Array] = []
    for name, (lo, hi) in LINEAR_BOUNDS.items():
        x = df[name].cast(pl.Float64).to_numpy()
        cols.append(np.clip((x - lo) / (hi - lo), 0.0, 1.0))
    for name in LOG_COLUMNS:
        x = df[name].cast(pl.Float64).to_numpy()
        cols.append(np.clip(np.log1p(np.clip(x, 0, TOP_CODE)) / math.log1p(TOP_CODE),
↪   0.0, 1.0))
    for name, values in CATEGORIES.items():
        column = df[name].to_list()
        for value in values:
            cols.append(np.array([1.0 if v == value else 0.0 for v in column]))
    return np.column_stack(cols)
```

The two techniques we compare are also fixed in advance. The first is private stochastic gradient descent (SGD), known as **DP-SGD**, the standard way to train a model with DP. It trains a logistic regression, as a bank would without DP, on small random batches. At each step it caps how much each customer can move the model, which is called *clipping*, and it adds random noise to the update (Opacus).

Opacus's privacy accountant sets the noise so that the whole run meets the target ε at $\delta = 10^{-5}$. From ε, δ, the share of customers in each batch and the number of passes, it returns the noise multiplier, the size of the noise relative to the clipping norm:

```
def noise_multiplier(eps: float, n: int) -> float:
    """The noise multiplier that makes DP-SGD with these settings (eps, DELTA)-DP, by
     ↪  Opacus's PRV accountant."""
    return float(
        get_noise_multiplier(
            target_epsilon=eps, target_delta=DELTA, sample_rate=BATCH / n,
 ↪  epochs=EPOCHS, accountant="prv"
        )
    )
```

Opacus then wraps the model, the optimizer and the data loader, so that every step draws a random batch, clips each customer's contribution and adds that noise. The rest of the loop is the same as without DP:

```
optimizer = torch.optim.SGD(layer.parameters(), lr=LR, momentum=MOMENTUM)
engine: PrivacyEngine | None = None
model: nn.Module = layer
if sigma is not None:
    engine = PrivacyEngine(accountant="prv")
    model, optimizer, loader = engine.make_private(  # Poisson sampling, per-example
 ↪  clipping, Gaussian noise
        module=layer, optimizer=optimizer, data_loader=loader, noise_multiplier=sigma,
 ↪  max_grad_norm=CLIP
    )
```

DP-SGD's settings were fixed in advance, with batches of about 1,000 customers, 10 passes over the data, a clipping norm of 1 and a learning rate of 0.5. The second technique is a **private naive Bayes** classifier from IBM's diffprivlib, which adds noise to simple averages per class (diffprivlib). It gives the stronger pure ε guarantee, with no δ. A pure guarantee also meets the $(\varepsilon, 10^{-5})$ standard, so both techniques sit on the same axis.

The experiment trains each technique at ten values of ε from 0.01 to 10. DP training is random, and one run can be lucky or unlucky, so each setting runs with 20 random seeds. Each model is also trained without DP, with the same settings, so that the figures show the cost of privacy directly. The script spreads these runs over the processor cores:

```
jobs: list[tuple[str, float | None, int, int]] = []
for technique in COLOURS:
    jobs += [(technique, eps, TRAIN_ROWS, seed) for eps in [None, *EPSILONS] for seed
 ↪  in range(seeds)]
for n in SIZES[:-1]:
    jobs += [("DP-SGD", eps, n, seed) for eps in [None, *EPSILONS] for seed in
 ↪  range(size_seeds)]
with ProcessPoolExecutor(max_workers=WORKERS, initializer=_init, initargs=(data,)) as
 ↪  pool:
    runs = list(pool.map(evaluate, jobs, chunksize=4))
```

5 Step 4: read the results, technical first

Figure 1 shows the results in the engineer's terms. Its vertical axis is the AUC, the probability that the model scores a randomly chosen eligible customer above a randomly chosen ineligible one, so 0.5 is guessing and 1 is perfect. The bottom axis is ε, and the top axis gives the attacker's belief ceiling from Section 3. The shaded bands run from the 10th to the 90th percentile over the seeds.

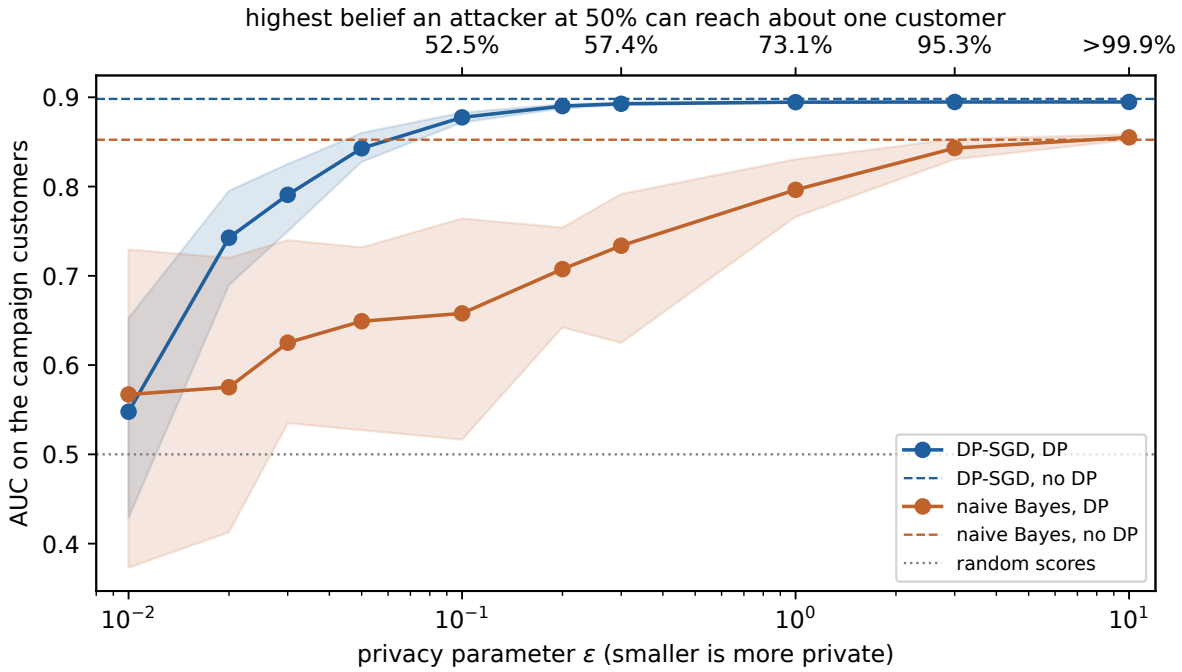


Figure 1: Model quality against the privacy setting. Each line is a DP technique, averaged over 20 seeds, with its 10th–90th percentile band; each dashed line is the same model trained without DP.

DP-SGD is better than naive Bayes at every privacy level except the smallest, $\varepsilon = 0.01$, where the two are almost tied, so naive Bayes can be dropped. DP-SGD's curve is also flat over most of the range, so from about $\varepsilon = 0.3$ upwards it is almost as good as the model trained without DP. Its quality only falls away below $\varepsilon \approx 0.1$, where the attacker's ceiling is already within a few points of 50%.

Figure 2 shows them in the bank's terms. Its vertical axis is the number of eligible customers reached by the 3,846 offers. The upper dashed line is the best model without DP, the dotted line is no model at all, and the shaded region on the right is what the privacy policy forbids.

The gap between a DP curve and the upper dashed line is the cost of privacy, counted in customers. The gap between the curve and the dotted line is what the model is worth compared with random offers. Table 3 lists every private configuration. A configuration is on the *frontier* when no other configuration is at least as private and reaches more customers, so only the frontier is worth considering. Here the frontier is DP-SGD at every level from $\varepsilon = 0.02$ up, with naive Bayes only at $\varepsilon = 0.01$.

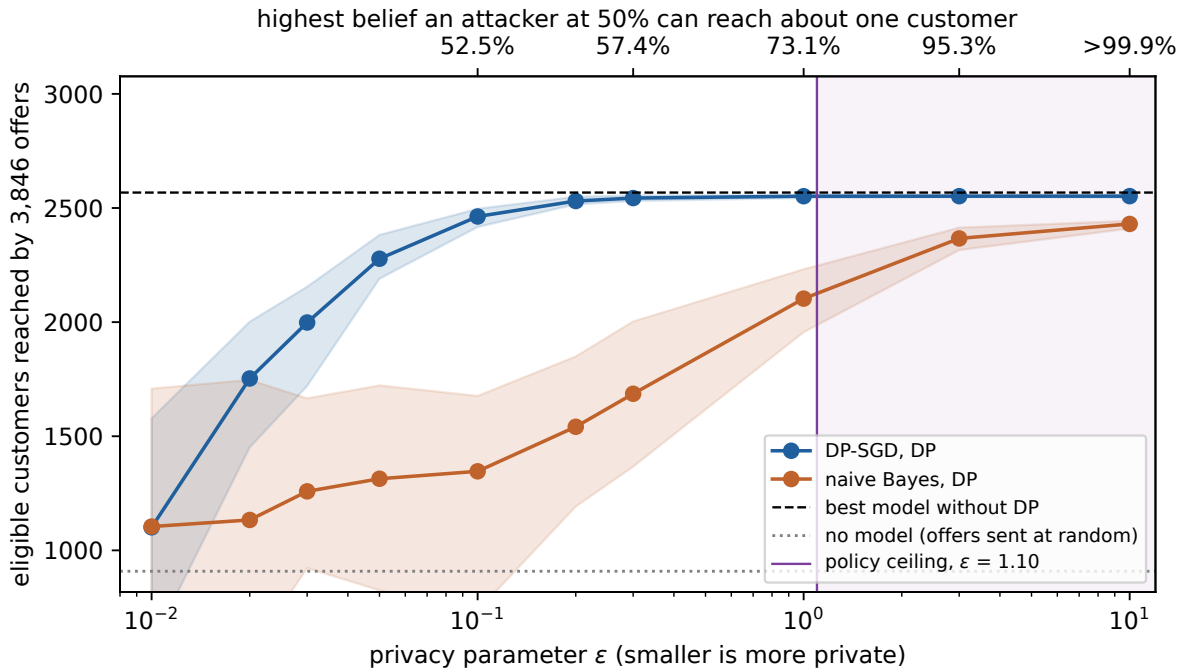


Figure 2: Eligible customers reached by the campaign against the privacy setting. The shaded region is ruled out by the policy ceiling.

The script finds the frontier with a short filter:

```
def non_dominated(points: list[tuple[float, float]]) -> list[int]:
    """Indices of the (privacy loss, utility) points that no other point dominates."""
    order = sorted(range(len(points)), key=lambda i: (points[i][0], -points[i][1]))
    frontier: list[int] = []
    best = float("-inf")
    for i in order:
        if points[i][1] > best:
            frontier.append(i)
            best = points[i][1]
    return frontier
```

Table 3: Every private configuration, averaged over the seeds.

Technique	ε	Attacker's belief ceiling	AUC	Eligible reached	On the frontier
DP-SGD	0.01	50.2%	0.548	1,101 of 3,846 (29%)	
naive Bayes	0.01	50.2%	0.567	1,104 of 3,846 (29%)	yes
DP-SGD	0.02	50.5%	0.743	1,753 of 3,846 (46%)	yes

Technique	ε	Attacker's belief ceiling	AUC	Eligible reached	On the frontier
naive Bayes	0.02	50.5%	0.575	1,133 of 3,846 (29%)	
DP-SGD	0.03	50.7%	0.791	1,998 of 3,846 (52%)	yes
naive Bayes	0.03	50.7%	0.625	1,259 of 3,846 (33%)	
DP-SGD	0.05	51.2%	0.843	2,278 of 3,846 (59%)	yes
naive Bayes	0.05	51.2%	0.649	1,314 of 3,846 (34%)	
DP-SGD	0.1	52.5%	0.878	2,463 of 3,846 (64%)	yes
naive Bayes	0.1	52.5%	0.658	1,346 of 3,846 (35%)	
DP-SGD	0.2	55.0%	0.890	2,531 of 3,846 (66%)	yes
naive Bayes	0.2	55.0%	0.708	1,542 of 3,846 (40%)	
DP-SGD	0.3	57.4%	0.893	2,544 of 3,846 (66%)	yes
naive Bayes	0.3	57.4%	0.734	1,686 of 3,846 (44%)	
DP-SGD	1	73.1%	0.895	2,552 of 3,846 (66%)	yes
naive Bayes	1	73.1%	0.796	2,103 of 3,846 (55%)	
DP-SGD	3	95.3%	0.895	2,552 of 3,846 (66%)	yes
naive Bayes	3	95.3%	0.843	2,367 of 3,846 (62%)	
DP-SGD	10	100.0%	0.895	2,552 of 3,846 (66%)	yes
naive Bayes	10	100.0%	0.855	2,430 of 3,846 (63%)	

6 Step 5: check how much the data size matters

The flat curve in Figure 2 depends on the bank having many customers. DP hides each person with noise, and the more customers there are, the less that noise matters next to their combined signal. So a bank with fewer customers, or a campaign aimed at one region, pays more for the same guarantee. To measure how much, the script trains DP-SGD again on the first 2,000 and the first 8,000 customers, and Figure 3 compares the three sizes.

With fewer customers the curve bends down at a larger ε. The gap to the model without DP is

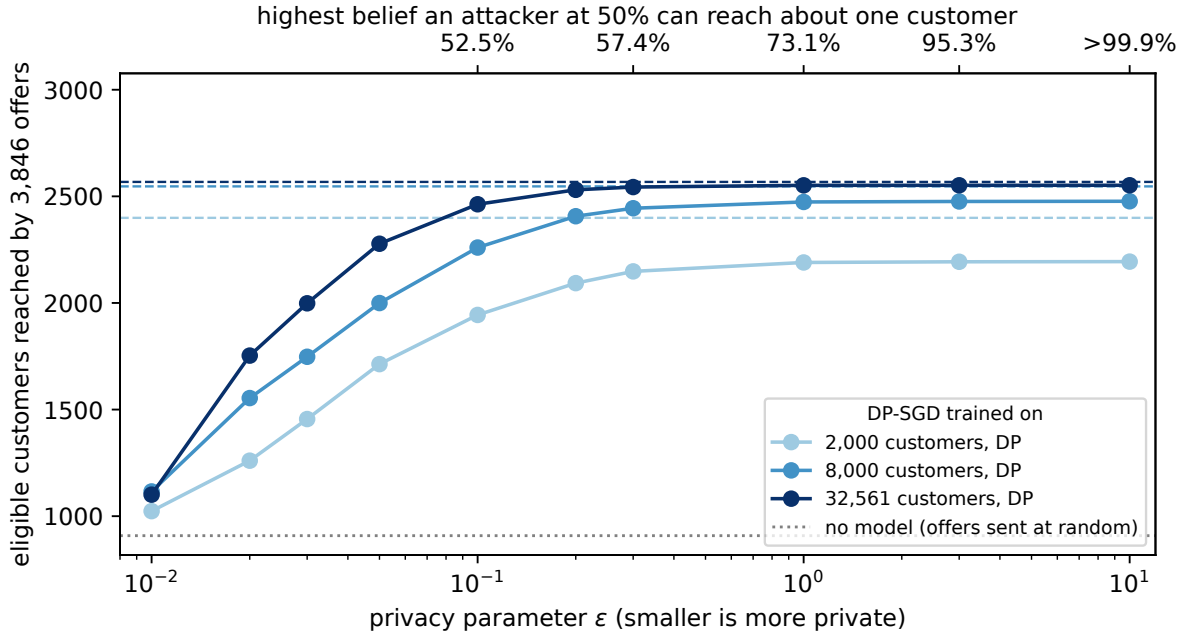


Figure 3: Eligible customers reached by DP-SGD trained on 2,000, 8,000 and 32,561 customers. Each dashed line is the same size without DP.

also wider at every setting, as Table 4 shows in numbers. For the smallest base the gap stays open even at $\varepsilon = 10$, where the noise is negligible. That part of the cost comes from the clipping settings, which suit the full dataset. So a bank in that position would first tune them on public or synthetic data. For a business this is often the most useful finding. It says whether the guarantee is cheap for this dataset, and whether more data would make a stricter policy affordable.

Table 4: Eligible customers missed because of privacy, by the number of customers the model is trained on.

Customers trained on	Reached without DP	Missed at $\varepsilon = 0.1$	Missed at $\varepsilon = 0.3$	Missed at $\varepsilon = 1$
2,000	2,399	455	251	209
8,000	2,547	287	102	73
32,561	2,568	105	24	16

7 Step 6: present the menu and decide

The decision maker needs a few options, each with a privacy sentence and its business effect, and Table 5 gives them. The options are the best technique at a few privacy levels, plus the two reference points, no model and no DP. The membership column gives a second privacy sentence. It bounds how often an attack can correctly identify a real customer, for any attack that wrongly accuses 1% of non-customers (Kairouz et al., 2015).

Table 5: The options menu for one campaign of 3,846 offers, best technique at each option.

Option	Attacker's belief ceiling	Member-ship TPR bound at 1% FPR	Eligible reached	Offers wasted	Missed vs. no DP	Gained vs. no model
No model (random offers)	—	—	909	2,937	1,659	0
$\varepsilon = 0.1$ (DP-SGD)	52.5%	1.1%	2,463	1,383	105	1,554
$\varepsilon = 0.3$ (DP-SGD)	57.4%	1.4%	2,544	1,302	24	1,635
$\varepsilon = 1$ (DP-SGD)	73.1%	2.7%	2,552	1,294	16	1,643
$\varepsilon = 3$ (DP-SGD)	95.3%	20.1%	2,552	1,294	16	1,643
No DP (the same logistic regression, no clipping or noise)	no limit	no limit	2,568	1,278	0	1,659

The policy ceiling from Section 3, $\varepsilon \leq 1.10$, picks the option.

The largest value on the grid below the ceiling is $\varepsilon = 1$, and at that value the DP-SGD model is on the frontier. Across the seeds it reaches between 2,544 and 2,557 of the 3,846 eligible customers, 2,552 on average. The model without DP reaches 2,568, and random offers reach 909. So privacy costs about 16 eligible customers per campaign, while the private model still reaches about 1,643 more than sending the offers at random. In money, with V the value to the bank of reaching one eligible customer, the cost of privacy is about $16\,V$ per campaign and the value of the private model over no model is about $1,643\,V$. Even at $\varepsilon = 10$, where the noise is small, the private model reaches 2,552. So almost all of the cost at the ceiling comes from clipping each customer's influence, which DP-SGD does at every ε, and almost none from the noise that the stricter policy adds.

The decision then comes down to one comparison. The finance team supplies V, the value of one eligible customer reached, so the cost of privacy in money is V times the customers it loses. The bank compares that cost with what the policy buys. The bank can hand the model to an outside agency, and it can answer a regulator or a customer who asks what the model reveals. Here the cost is small, so the comparison is easy, but Table 4 shows how quickly that changes with fewer customers.

Three people sign the decision. The privacy officer signs the ceiling, fixed before the results were known. The engineer signs that the configuration meets the ceiling, with the accountant's ε and δ and the settings of Section 4. The business owner signs the cost in customers, and so in money. A

release record keeps all of this with the script, so that anyone can check the decision and revisit it when the data or the policy changes.

8 Doing this in another business

The six steps carry over to any use case, and only their contents change. The metric is whatever count the decision turns on, such as fraud caught at a fixed number of reviews, and the baseline is what the business does without the model. The ceiling is still a sentence about an attacker's belief, fixed before anyone sees the results. The output is still a short menu with a cost in the business's own units, for the people who own the risk and the revenue to sign.

9 Sources

- Becker, B. and Kohavi, R. (1996). Adult. UCI Machine Learning Repository. Creative Commons Attribution 4.0 licence.
- Abadi, M. et al. (2016). Deep Learning with Differential Privacy. *ACM CCS 2016*. The DP-SGD algorithm.
- Yousefpour, A. et al. (2021). Opacus: User-Friendly Differential Privacy Library in PyTorch. The library and its privacy accountant.
- Holohan, N., Braghin, S., Mac Aonghusa, P. and Levacher, K. (2019). Diffprivlib: The IBM Differential Privacy Library.
- Kairouz, P., Oh, S. and Viswanath, P. (2015). The Composition Theorem for Differential Privacy. *ICML 2015*. The hypothesis-testing reading of (ε, δ) behind the membership bound.
- Nanayakkara, P. et al. (2023). What Are the Chances? Explaining the Epsilon Parameter in Differential Privacy. *USENIX Security 2023*. Evidence that odds-based explanations of ε work best.
- Papernot, N. and Steinke, T. (2022). Hyperparameter Tuning with Rényi Differential Privacy. *ICLR 2022*. Why tuning on private data leaks.
- Ponomareva, N. et al. (2023). How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy. *Journal of Artificial Intelligence Research* 77.
- Near, J., Darais, D., Lefkovitz, N. and Howarth, G. (2025). Guidelines for Evaluating Differential Privacy Guarantees. NIST SP 800-226.